metroLog — шпаргалка к защите (РКСП)

Проект: metroLog — система учёта средств измерений (СИ) и эталонов (ЭСИ) **Репозиторий:** https://github.com/mflkee/metroLog · **Стенд:** metrolog.mkair-it.ru **Студент:** Макеев Глеб Борисович, BHEBDPS-24-RKSP-5

---

1. Карточка проекта (держи в голове)

Что	Значение
Назначение	единый реестр оборудования, контроль сроков поверки, ремонты, интеграция с ФГИС «Аршин»
Пользователи	разработчик, администратор, сотрудник метрологической службы (MKAIR), заказчик
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic, Pydantic
БД	PostgreSQL (+ Redis для очереди задач, RQ)
Frontend	React 19 + TypeScript, Vite, TanStack Query, Zustand, Tailwind
Инфраструктура	Docker Compose, nginx, GitHub Actions (GHCR)
API	**94 пути / 127 операций**, OpenAPI/Swagger на `/docs`
БД-схема	**30 сущностей**, **66 миграций** Alembic
Тесты	**229 pytest** (14 файлов) + 33 файла vitest
Нагрузка	locust: 30 и 60 пользователей, 0 ошибок

Цифры, которые называют вслух: 30 сущностей, 66 миграций, 94/127 эндпоинтов, 229 тестов, 2 протокола (HTTP/REST + SMTP).

---

2. Рассказ на 60 секунд

> metroLog — система учёта средств измерений для метрологической службы. Раньше реестр вёлся в Excel по папкам, сроки поверки контролировали вручную, данные из ФГИС «Аршин» копировали руками. Сейчас это клиент-серверное приложение: SPA на React, REST API на FastAPI, PostgreSQL, фоновая очередь на Redis. Есть иерархия «папка → группа → оборудование», поверки и ремонты с этапами, импорт из «Аршина», комментарии и вложения, email-уведомления, аудит и выгрузка в XLSX. Доступ разграничен по ролям и по папкам, при отсутствии прав сервер отвечает 404, чтобы не раскрывать существование данных.

---

3. ТЗ: что важно помнить

AS IS: Excel-файлы по подразделениям, ручной контроль сроков, ручной ввод из «Аршина», история в переписке, нет разграничения доступа.

ТО ВЕ: единый реестр, автоматический импорт и пакетное обновление из «Аршина», процессы поверки/ремонта с этапами и сроками, комментарии и уведомления, роли + доступ по папкам, журнал событий, дашборд, экспорт XLSX.

Границы (спросят почти наверняка): - **В системе:** реестр СИ/ЭСИ/ИО/ВО, поверки, ремонты, Аршин, коммуникации, роли, отчёты. - **Вне системы:** бухгалтерия и 1С, мобильное приложение, закупки, выпуск бумажных свидетельств.

Роли: `DEVELOPER` (диагностика), `ADMINISTRATOR` (пользователи, права, справочники), `MKAIR` (ведение реестра, поверки, ремонты), `CUSTOMER` (просмотр своих папок, комментарии).

---

4. Архитектура

```
Браузер (React 19 SPA)
   │ HTTPS / REST (JSON)
   ▼
nginx :80 ── статика SPA + reverse proxy /api/ → backend:8000
   │
   ▼
FastAPI (uvicorn, 2 воркера)
   ├── PostgreSQL 16 (30 таблиц)
   ├── Redis + RQ (email-уведомления в фоне)
   ├── ФГИС «Аршин» (HTTP API, httpx)
   └── SMTP (email)
```

Почему так: - **SPA + REST** — один API на веб и будущие клиенты; nginx отделяет статику от API. - **Очередь (RQ)** — отправка писем не должна блокировать HTTP-запрос: тяжёлое выносим в фон. - **PostgreSQL** — связи M:N, частичные индексы, JSON-поля; транзакции для операций с оборудованием. - **Миграции Alembic** — схема версионируется и применяется автоматически при старте.

---

5. Аутентификация и доступ

- **Свой токен, не JWT:** base64url(payload) + подпись **HMAC-SHA256**, payload — `sub`, `role`, `iat`, `exp`. Токен stateless: сервер не хранит сессии, проверка = пересчёт подписи.
- **Почему не JWT:** не нужен весь JWT-набор (алгоритмы, kid, JWKS), проще контролировать формат; отзыв — ротацией `secret_key` (все сессии разом).
- **Пароли:** PBKDF2-HMAC-SHA256, **600 000 итераций**, соль на пользователя; временный пароль при выдаче учётки + смена при первом входе.
- **Права:** роль + список разрешённых/скрытых папок. Нет доступа к объекту → **404 вместо 403** (soft-denial: не раскрываем, что объект существует).
- **Где смотреть:** `app/utils/security.py` (токен, хеши), `app/api/deps.py` (текущий пользователь, проверка ролей), `app/services/equipment_service.py` (проверка доступа к папке).

---

6. База данных

30 сущностей, основные группы: - реестр: `users`, `equipment_folders`, `equipment_groups`, `equipment`, `deadline_presets`; - процессы: `verifications`, `si_verifications`, `repairs`, `equipment_process_templates`; - коммуникации: `equipment_comments`, `*_messages`, `*_attachments`, `*_subscriptions`; - задачи: `tasks`, `task_checklist_items`, `task_participants`, `task_reminders`; - служебное: `event_logs`, `equipment_folder_refresh_tasks/rows`, `auth_codes`.

Что помнить: - иерархия «папка → группа → оборудование» — 1:N на каждом уровне; - подписки на процессы — M:N через промежуточные таблицы (`*_subscriptions`); - **66 миграций** — история схемы, накатывается `alembic upgrade head`; - нормализация до 3НФ; осознанные денормализации — снимки названий в истории (например, `deadline_preset_name`) и агрегаты для дашборда; - индексы: частичные и составные под фильтры списков (80 индексов в выгрузке схемы).

---

7. API

REST, JSON, версия в пути: `/api/v1/...`. Документация — `/docs` (Swagger) и `slumber.yml`.

Что	Как
Списки	пагинация `page` / `per_page` (и совместимые `limit` / `offset`), в ответе `total`, `total_pages`
Сортировка	`sort_key` + `sort_direction`, поле — из белого списка (Literal-тип, иначе 422)
Фильтры	папка, группа, статус, тип, объект, местоположение + поиск `query`
Поиск	термовый ILIKE по нескольким колонкам (`app/repositories/search.py`), термы могут быть в разных полях
Ошибки	единый формат `{detail, error_code, field}`; коды `NOT_FOUND`, `CONFLICT`, `FORBIDDEN`, `VALIDATION_ERROR`, `INTERNAL_ERROR`
Статистика	`/api/v1/stats/summary`, `/stats/equipment?group_by=status|equipment_type|folder_id`, `/stats/verifications?group_by=status` — с учётом прав на папки
Экспорт	XLSX: оборудование, поверки, ремонты, журнал событий
Служебное	`/health`, `/health/ready`

Второй протокол — SMTP: email-уведомления (упоминания, выдача пароля, изменения статусов, проверка SMTP отдельным эндпоинтом). HTTP/REST — основной.

---

8. Тесты и нагрузка

- **229 pytest**: аутентификация и права, CRUD оборудования, поверки/ремонты, Аршин (с моками), формат ошибок, статистика (включая изоляцию по папкам), пагинация, health.
- **33 файла vitest** на фронте (компоненты, хуки, API-клиент).
- **locust**, read-профиль:

Прогон	Запросов	Ошибок	Медиана	p95	RPS
30 users	2 311	0	53 мс	1.4 с	19.3
60 users	3 255	0	660 мс	2.4 с	27.3

Вывод из нагрузки (сказать честно): при удвоении нагрузки медиана выросла в 12 раз — узкое место найдено: `POST /auth/login`, PBKDF2 с 600 000 итераций выполняется в event loop и блокирует обработку остальных запросов. Правильное решение — вынести хеширование в пул потоков (`anyio.to_thread.run_sync`), но ослаблять 600k итераций нельзя.

---

9. Код: где что искать

```
backend/app/
  api/v1/routes/     equipment, users, tasks, events, arshin, health, stats
  api/deps.py        текущий пользователь, проверки ролей
  services/          бизнес-логика (equipment_*, user_service, arshin_service, stats_service)
  repositories/      запросы к БД (SQLAlchemy), поиск, агрегаты
  models/            SQLAlchemy-модели (30 сущностей)
  schemas/           Pydantic: запросы/ответы, ErrorResponse
  core/              конфиг, ошибки, обработчики
  tasks/             Redis + RQ (очередь уведомлений)
  utils/             токены, хеши, экспорт XLSX
frontend/src/        pages, components, api (Axios), hooks (TanStack Query)
```

---

10. Вопросы, которые задают, и короткие ответы

1. **Что делает система?** — реестр СИ/ЭСИ, поверки, ремонты, Аршин, уведомления, отчёты.
2. **Почему FastAPI, а не Django/Spring?** — асинхронный, Pydantic-валидация из коробки, OpenAPI генерируется сам; для API-центричного сервиса этого достаточно.
3. **Почему свой токен, а не JWT?** — нужен stateless-токен с ролью; JWT тянет за собой алгоритмы/kid/JWKS, а отзыв всё равно через секрет. Свой формат проще проверять.
4. **Где хранятся пароли?** — PBKDF2-HMAC-SHA256, 600k итераций, соль на пользователя, в БД только хеш.
5. **Что при отсутствии прав?** — 404, а не 403: не раскрываем существование объекта.
6. **Как ограничен доступ к папкам?** — у пользователя списки разрешённых и скрытых папок; каждый запрос к оборудованию проверяет папку.
7. **Как устроена пагинация?** — `page` / `per_page` (по умолчанию 20, максимум 200), `limit` / `offset` как совместимый вариант; конфликт параметров → 422; в ответе `total`, `total_pages`.
8. **Почему сортировка по белом списку?** — имя поля из запроса нельзя подставлять в `ORDER BY`: это SQL-инъекция; поле валидируется enum-типом.
9. **Как ищет поиск?** — разбивает строку на термы, каждый ищется через ILIKE по нескольким колонкам, термы могут быть в разных полях («Мкаир Кужим» находится).

10. **Зачем очередь Redis?** — письма отправляются в фоне, HTTP-запрос не ждёт SMTP.
11. **Как обеспечивается целостность?** — транзакции на операции, FK с политиками удаления, миграции Alembic, инварианты в сервисах (например, «один активный ремонт»).
12. **Что с безопасностью файлов?** — ограничение размера, белый список типов, хранение вне статики, отдача через API с проверкой прав.
13. **Как проверяли нагрузку?** — locust, read-профиль, 30 и 60 пользователей; нашли узкое место в логине.
14. **Второй протокол?** — SMTP. WebSocket/SSE не реализованы (честно).
15. **Что бы улучшил?** — вынести PBKDF2 в пул потоков, добавить SSE для уведомлений в UI, кеш дашборда, e2e-тесты.
16. **Как тестировали?** — pytest с фикстурами на SQLite/in-memory и моками Аршина; на фронте vitest.
17. **Как деплоится?** — Docker Compose, nginx, self-hosted GitHub Actions: сборка образов в GHCR.
18. **Что самое сложное было?** — интеграция с «Аршином» (нестабильный внешний API) и права по папкам сквозь все запросы.
19. **Где бизнес-логика?** — в `services/`; роуты только принимают запрос и вызывают сервис.
20. **Что если внешний сервис упал?** — таймауты, ретраи, ошибка 502 с `error_code`, локальные данные не теряются.

---

11. Что сказать, если спросят про слабые места (честность = плюс)

1. **Логин медленный под нагрузкой** — PBKDF2 в event loop; решение известно, не применено.
2. **Второй протокол — SMTP**, а не WebSocket; реалтайм-обновления в UI не сделаны.
3. **Фронтенд** покрыт частично (33 файла тестов), e2e нет.
4. **Нет горизонтального масштабирования** приложения из коробки (несколько воркеров без sticky-сессий работают, потому что токены stateless — но очередь и файлы требуют общего хранилища).
5. **Мобильного клиента нет** — по ТЗ вне границ.

---

12. Мини-шпаргалка «одной строкой»

- 30 сущностей · 66 миграций · 94 пути / 127 операций · 229 pytest · 33 vitest
- HMAC-SHA256, PBKDF2 600k, TTL токена, 404 вместо 403
- пагинация page/per_page + limit/offset, сортировка по enum, поиск по термам
- ошибки `{detail, error_code, field}`; `stats/*` с правами; XLSX-экспорт
- SMTP = второй протокол; узкое место — логин (PBKDF2 в event loop)